UNIVERSITY OF THESSALY
SCHOOL OF ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Detection of Suspicious Bitcoin Transactions using Machine Learning Methods

Special Topic

Eleni Tsitoni

Supervisor: Eleni Tousidou

February 2026

UNIVERSITY OF THESSALY
SCHOOL OF ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Detection of Suspicious Bitcoin Transactions using Machine Learning Methods

Special Topic

Eleni Tsitoni

Supervisor: Eleni Tousidou

February 2026

ΠΑΝΕΠΙΣΤΗΜΙΟ ΘΕΣΣΑΛΙΑΣ
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Ανίχνευση Ύποπτων Συναλλαγών Bitcoin με Μεθόδους Μηχανικής Μάθησης

Ειδικό Θέμα

Ελένη Τσιτώνη

Επιβλέπουσα: Ελένη Τουσίδου

Φεβρουάριος 2026

Special Topic

Detection of Suspicious Bitcoin Transactions using Machine Learning Methods

Eleni Tsitoni

Abstract

Detecting fraud in cryptocurrency transactions is difficult because blockchain networks are complex and there isn't enough labeled data. Our project explores how machine learning and graph neural networks (GNNs) can help detect fraud in Bitcoin transactions. We use the Elliptic and Elliptic++ datasets. Our main goals are to reduce the need for manual data labeling through Active Learning and to improve fraud detection accuracy with Heterogeneous Graph Neural Networks.

The research looks at Bitcoin data in two forms: One as a simple transaction graph and the other as a mixed wallet-transaction network. We do this to capture different layers of blockchain activity. In the Active Learning experiments, a Random Forest trained on just 10% of the labeled data reached the same F1-score as one trained on the entire dataset, requiring 90% fewer annotations. We used deep learning models like LSTM and GCN to compare how well they work on the homogeneous graph.

Then we compare several graph-based models and we find that heterogeneous GNNs, especially the Heterogeneous Graph Transformer (HGT), give the best results. On the Elliptic++ dataset HGT achieves a 0.90 recall, outperforming both traditional machine learning methods and standard (homogeneous) GCNs. These results highlight that wallet-transaction relationships uncover fraud patterns that simpler models often miss.

In short, combining Active Learning with Heterogeneous GNNs offers an efficient and effective approach to fraud detection. The method is suitable for real-world anti-money laundering systems and may extend to other blockchain networks.

Keywords:

Machine Learning, Graph Neural Networks, Bitcoin, Fraud Detection, Active Learning, Heterogeneous Graph Transformer

Ειδικό Θέμα

Ανίχνευση Ύποπτων Συναλλαγών Bitcoin με Μεθόδους Μηχανικής Μάθησης

Ελένη Τσιτώνη

Περίληψη

Η ανίχνευση απάτης σε συναλλαγές κρυπτονομισμάτων είναι δύσκολη επειδή τα δίκτυα blockchain είναι πολύπλοκα και δεν υπάρχουν αρκετά επισημασμένα δεδομένα. Το έργο μας εξερευνά πώς η μηχανική μάθηση και τα γραφητικά νευρωνικά δίκτυα (GNNs) μπορούν να βοηθήσουν στην ανίχνευση απάτης σε συναλλαγές Bitcoin. Χρησιμοποιούμε τα σύνολα δεδομένων Elliptic και Elliptic++. Οι κύριοι στόχοι μας είναι να μειώσουμε την ανάγκη για χειροκίνητη επισήμανση δεδομένων μέσω Ενεργής Μάθησης (Active Learning) και να βελτιώσουμε την ακρίβεια ανίχνευσης απάτης με Ετερογενή Γραφητικά Νευρωνικά Δίκτυα (Heterogeneous Graph Neural Networks).

Η έρευνα εξετάζει τα δεδομένα του Bitcoin με δύο μορφές: τη μία ως απλό γράφημα συναλλαγών και την άλλη ως μικτό δίκτυο πορτοφολιών–συναλλαγών. Το κάνουμε αυτό για να αποτυπώσουμε διαφορετικά επίπεδα δραστηριότητας του blockchain. Στα πειράματα Ενεργής Μάθησης, ένα μοντέλο Random Forest που εκπαιδεύτηκε μόνο με το 10

Στη συνέχεια, συγκρίνουμε διάφορα μοντέλα βασισμένα σε γράφους και διαπιστώνουμε ότι τα ετερογενή GNNs, ειδικά ο Heterogeneous Graph Transformer (HGT), δίνουν τα καλύτερα αποτελέσματα. Στο σύνολο δεδομένων Elliptic++, το HGT επιτυγχάνει ανάκληση (recall) 0.90, ξεπερνώντας τόσο τις παραδοσιακές μεθόδους μηχανικής μάθησης όσο και τα τυπικά (ομογενή) GCNs. Αυτά τα αποτελέσματα δείχνουν ότι οι σχέσεις πορτοφολιού–συναλλαγής αποκαλύπτουν μοτίβα απάτης που τα απλούστερα μοντέλα συχνά παραβλέπουν.

Συνοπτικά, ο συνδυασμός Ενεργής Μάθησης με Ετερογενή GNNs προσφέρει μια αποδοτική και αποτελεσματική προσέγγιση για την ανίχνευση απάτης. Η μέθοδος είναι κατάλληλη για πραγματικά συστήματα καταπολέμησης ξεπλύματος χρήματος και μπορεί να επεκταθεί σε άλλα δίκτυα blockchain.

Λέξεις-κλειδιά:

Machine Learning, Graph Neural Networks, Bitcoin, Fraud Detection, Active Learning, Heterogeneous Graph Transformer

Table of contents

Abstract **7**

Περίληψη **8**

Table of contents **11**

List of figures **15**

List of tables **17**

1 Introduction **1**

1.1 Background 1

1.2 Problem Definition 1

1.3 Motivation 2

1.4 Research Objectives & Questions 2

1.5 Proposed Solution 3

1.6 Contributions 4

1.7 Project Organization 4

2 Related Work **5**

2.1 Datasets for Bitcoin Fraud Detection 5

2.2 Traditional Machine Learning Approaches 7

2.2.1 Supervised Baselines 7

2.2.2 Unsupervised Methods and Label Scarcity 8

2.3 Graph Neural Networks for Homogeneous Graphs 8

2.3.1 Graph Convolutional Networks 8

2.3.2 Sequential Models 9

2.4 Heterogeneous Graph Neural Networks 9
2.4.1 The Importance of Entity-Level Modeling 9
2.4.2 Heterogeneous Graph Attention Network (HAN) 10
2.4.3 Heterogeneous Graph Transformer (HGT) 10
2.5 Research Gaps 10

3 Methodology 13
3.1 Bitcoin Transaction Networks 13
3.1.1 The UTXO Model 13
3.1.2 Two Graph Representations 14
3.2 Research Framework 15
3.2.1 Design Science Research 15
3.2.2 Two-Track Experimental Design 16
3.3 Active Learning 16
3.3.1 The Cold Start Problem 16
3.3.2 The Mimicry Hypoproject 17
3.3.3 Query Strategies 17
3.3.4 Switching Policy 18
3.3.5 The Upper Bound 18
3.4 Model Architectures 19
3.4.1 Feature-Based Baselines 19
3.4.2 Sequential Models 19
3.4.3 Graph Neural Networks for Homogeneous Graphs 20
3.4.4 Heterogeneous Graph Neural Networks 20
3.5 Evaluation Metrics 22

4 Experimental Framework and Results 23
4.1 Overview 23
4.2 Common Experimental Protocol 23
4.2.1 Design Science Research Framework 24
4.2.2 Temporal Split Strategy 24
4.2.3 Feature Scaling 24
4.2.4 Handling Class Imbalance 24

4.2.5 Reproducibility Protocol 25
4.3 Experimental Track 1: Elliptic Dataset 25
4.3.1 Data Preprocessing 25
4.3.2 Supervised Baselines (Upper Bounds) 25
4.3.3 Active Learning Framework 28
4.3.4 Scenario 1: Pure Unsupervised 29
4.3.5 Scenario 2: Hybrid Active Learning 32
4.3.6 GCN 34
4.3.7 LSTM 36
4.4 Experimental Track 2: Elliptic++ Dataset 39
4.4.1 Data Preprocessing 39
4.4.2 Baseline Models 39
4.4.3 HAN 40
4.4.4 HGT 41
4.4.5 Comparison of All Models on Elliptic++ 42
4.4.6 Analysis and Discussion 43
4.4.7 Limitations 44

5 Conclusions **45**
5.1 Summary of Contributions 45
5.2 Answer to the Research Questions 46
5.3 Limitations 46
5.4 Future Work 46
5.5 Closing Remark 47

Bibliography **49**

List of figures

2.1 Example of an address–transaction graph from Elliptic++ (Elmougy & Liu [1]). Each transaction node (circles) is linked to wallet addresses (squares) that send or receive funds. The figure shows an illegal actor (blue nodes) at the start and shows how wallet connections reveal coordinated fraud that isn't visible in just the transaction graph. 6

2.2 Visualization of money flow transaction graphs from a sample of the Elliptic dataset (Weber et al. [2]). overall distribution of transactions at time step 32. The lower panels show focused subgraphs for different transaction types: unknown (left), illicit (middle), and licit (right). In these graphs, each node represents a Bitcoin transaction and the directed edges indicate the flow of funds between transactions. 7

2.3 Illustration of heterogeneous meta-paths used in HAN models (adapted from Wang et al. [3]). The upper part shows homogeneous address and transaction graphs, while the lower part introduces heterogeneous address–transaction relationships. Meta-paths such as Transaction–Address–Transaction (TAT) and Address–Transaction–Address (ATA) capture semantic connections between different node types. 10

4.1 Precision-Recall vs. Threshold for Random Forest. The best $F1$-Score (0.83) happens at a threshold of around ≈ 0.58. The model maintains high precision even when recall is higher. 26

4.2 Precision-Recall vs. Threshold for XGBoost. The model performs competitively with a maximum $F1$-Score of 0.82. 27

4.3 Precision-Recall vs. Threshold for Logistic Regression. The sharp drop in precision means the model isn't good at telling the classes apart. It's having trouble separating the positive and negative cases effectively.s (Max $F1 = 0.52$). 28
4.4 t-SNE projection of the Elliptic test set. Illicit transactions (red) are not spatially isolated but are mixed with licit transactions (green), confirming the Mimicry Effect. 31
4.5 Learning curves for Random Forest with Immediate Transition. Left: LOF warm-up. Right: Elliptic Envelope warm-up. 33
4.6 Comparison of GCN with state-of-the-art methods 35
4.7 LSTM training curves showing rapid convergence of accuracy (blue) and loss (red) over 13 epochs. 37
4.8 Training Recall curves for Heterogeneous GNN models on Elliptic++. HAN (orange) achieves near-total recall in early epochs due to class-weight bias, while HGT (green) demonstrates steadier convergence, reaching a maximum Recall of 0.9261. 43

List of tables

3.1 Comparison of Elliptic and Elliptic++ datasets 15

4.1 Supervised Baseline Performance and Optimized Thresholds on the Full Training Set. 26

4.2 Performance of Unsupervised Warm-up Strategies vs. Random Sampling (F1-Score statistics over 5 runs). 30

4.3 Optimal Results for Scenario 2 (Hybrid Active Learning) at 10% Labeling Budget. 32

4.4 GCN Performance with Temporal Split. 34

4.5 GCN Comparison with State-of-the-Art. 35

4.6 LSTM Training Convergence. 36

4.7 LSTM Performance on Test Set. 36

4.8 LSTM Comparison with Baselines. 38

4.9 Logistic Regression Performance on Elliptic++. 39

4.10 Random Forest Performance on Elliptic++. 40

4.11 HAN Training Progression. 41

4.12 HGT Training Progression. 41

4.13 Complete Model Comparison on Elliptic++. 42

4.14 Comparison with State-of-the-Art on Elliptic++. 42

Chapter 1

Introduction

1.1 Background

Bitcoin was created by Satoshi Nakamoto in 2008. It's a decentralized electronic payment system that operates without banks or central authorities. This innovation has completely changed the way we think about money and transactions.

Traditional transaction methods are no longer suitable. Criminals use clever techniques to hide their traces, making detection with simple rule-based systems increasingly difficult. This has driven the adoption of Artificial Intelligence and advanced machine learning algorithms for effective fraud detection.

Bitcoin has opened the door for illicit activities like money laundering, ransomware payments and Dark Web transactions because of it's pseudonymous nature and lack of central oversight. Every year, billions of dollars in "dirty money" are laundered through cryptocurrencies. Because of this, governments and crypto exchanges have started to implement stricter Anti-Money Laundering (AML) policies.

1.2 Problem Definition

This project addresses two fundamental challenges in Bitcoin fraud detection:

Label Scarcity

To train a machine learning model we need labeled training data, which are examples of transactions that have already been tagged as "licit" or "illicit." The problem is that these la-

bels are extremely rare and cost a lot of money. Each transaction must be manually reviewed by specialized analysts with deep expertise in fraud detection techniques. This scarcity of labeled data creates a “cold start problem”. This happens when we don’t have enough examples to even get the model started or help it learn effectively [4].

Relational Complexity

Bitcoin transactions do not happen in isolation. Actually, they form a complex network where money flows between wallets through chains of transactions. Traditional machine learning models (Logistic Regression, Random Forest) analyze each transaction independently and they ignore this network structure and the behavioral patterns it reveals.

1.3 Motivation

The goal of this study is to develop an Anti-Money Laundering system that is both *effective* in detecting fraud and *affordable* to run.

Efficiency through Active Learning

Labeling thousands of transactions randomly is an expensive and time-consuming process. That’s why we use Active Learning, which strategically picks the most useful samples for manual review.

Relational Reasoning through Heterogeneous Modeling

We use models that are specifically built to work with networks that have different types of entities. These models separate transactions from wallets, allowing them to detect sophisticated fraud patterns that basic approaches can’t see.

1.4 Research Objectives & Questions

This work uses the Design Science Research (DSR) methodology and tries to answer three main research questions:

- **RQ1 (Efficiency):** Can an Active Learning system match the performance of traditional supervised models while using only a small fraction of labeled data?
- **RQ2 (Architecture):** Do heterogeneous graph models (HAN, HGT) outperform feature-based baselines (LR, RF) when we have entity-level information available?
- **RQ3 (Bootstrap):** Can unsupervised anomaly detection effectively initialize the system when no labeled data are available?

1.5 Proposed Solution

This project uses a two-track experimental design to address label scarcity and the limitations of the architecture:

Track 1: Active Learning on Transaction Networks

We apply Active Learning to the homogeneous transaction graph to minimize labeling costs. We evaluate two scenarios:

- **Pure Unsupervised:** Uses only anomaly detection methods throughout the labeling process.
- **Hybrid Switching:** Starts with unsupervised warm-up, then transitions to supervised query strategies once initial labels are obtained.

The performance is compared to basic deep learning methods (GCN, LSTM) and fully-supervised models (RF, XGBoost, LR).

Track 2: Heterogeneous Models on Elliptic++

We compare feature-based baselines (Logistic Regression, Random Forest) against heterogeneous graph models (HAN, HGT) to see how helpful it is to look at individual entities when we have transaction and wallet data.

Detailed experimental protocols, model configurations, and evaluation procedures are described in Chapters **??** and **??**.

1.6 Contributions

This project offers three principal contributions:

- **Methodological:** We present a framework for designing and evaluating cost-effective AML systems using Active Learning.
- **Practical:** We show that labeling costs can be reduced by up to 90% while maintaining high detection accuracy.
- **Experimental:** We compare different model architectures on heterogeneous Bitcoin network data. The results prove that models which can tell entity types apart work way better than homogeneous approaches.

1.7 Project Organization

The work is organized as follows:

- **Chapter 1: Introduction:** Presents the problem, research objectives and our proposed solution.
- **Chapter 2: Related Work:** Examines literature in AML, GNNs and Active Learning to identify current research gaps.
- **Chapter 3: Methodology:** Introduces Bitcoin transaction networks, the Design Science Research framework, Active Learning theory (cold start problem, query strategies, switching policies), and the six model architectures (LR, RF, GCN, LSTM, HAN, HGT).
- **Chapter 4: Experiments and Results:** Details the experimental setup (data preprocessing, temporal split strategy, hyperparameter configurations, evaluation metrics), presents learning curves and performance comparisons, and interprets why heterogeneous models (HGT, HAN) excel at capturing entity-level relationships.
- **Chapter 5: Conclusions:** Summarizes findings, answers research questions, and proposes future directions.

Chapter 2

Related Work

This chapter reviews existing research on Bitcoin fraud detection. We organize the literature by methodology. We start with traditional machine learning approaches and then we move to graph-based methods for homogeneous networks. Finally we look at recent work on heterogeneous graph models. The chapter ends by pointing out the specific gaps that led us to do this project.

2.1 Datasets for Bitcoin Fraud Detection

Initial studies on cryptocurrency fraud were limited because there was a significant lack of data. Specifically, there were no large labeled datasets available for everyone to use. This changed when Weber et al. [2] published the **Elliptic dataset**, which is a graph of Bitcoin transactions that includes verified labels identifying illegal and legal activities.

However, Elmougy and Liu [1] identified an important limitation: The Elliptic datasets only shows transaction-to-transaction flows and it doesn't include wallet addresses. To fix this, they released **Elliptic++**. They added wallet nodes and different types of edges between wallets and transactions to the original dataset. This allows fraud analysis at both transaction and account levels, which is important for detecting coordinated illegal activities.

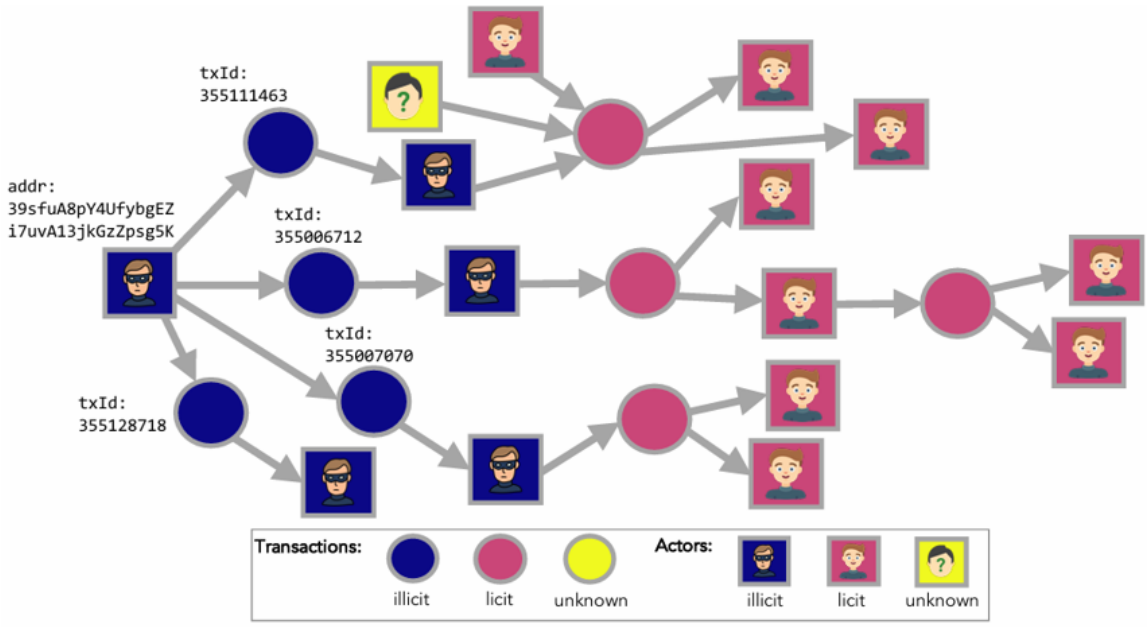

Figure 2.1: Example of an address–transaction graph from Elliptic++ (Elmougy & Liu [1]). Each transaction node (circles) is linked to wallet addresses (squares) that send or receive funds. The figure shows an illegal actor (blue nodes) at the start and shows how wallet connections reveal coordinated fraud that isn't visible in just the transaction graph.

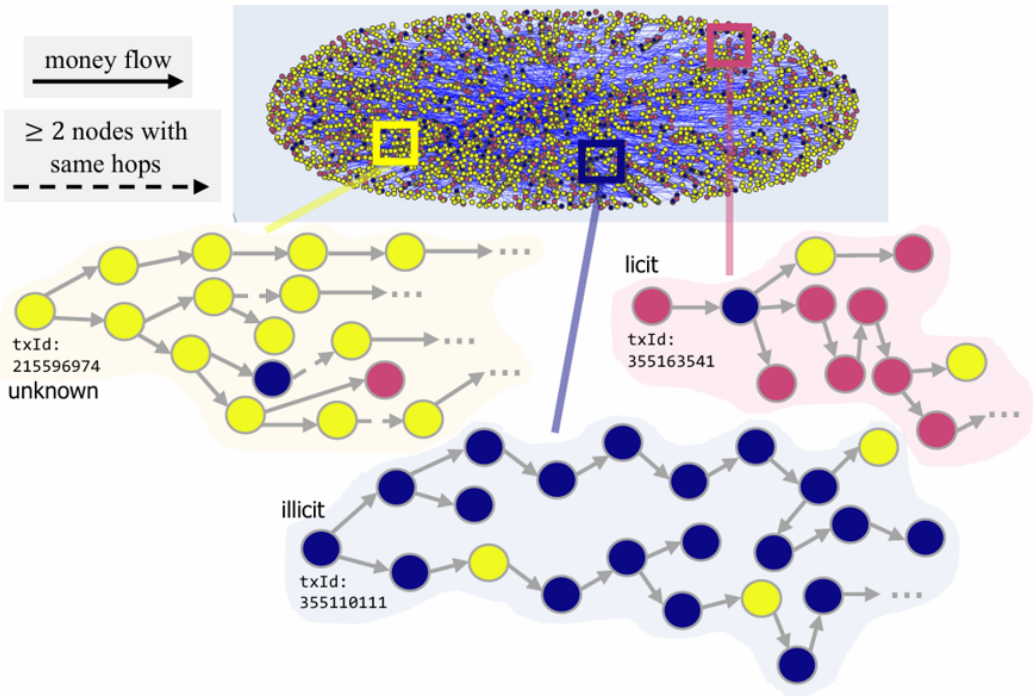

Figure 2.2: Visualization of money flow transaction graphs from a sample of the Elliptic dataset (Weber et al. [2]). overall distribution of transactions at time step 32. The lower panels show focused subgraphs for different transaction types: unknown (left), illicit (middle), and licit (right). In these graphs, each node represents a Bitcoin transaction and the directed edges indicate the flow of funds between transactions.

2.2 Traditional Machine Learning Approaches

Initial fraud detection systems relied on feature-based models that treat each transaction independently.

2.2.1 Supervised Baselines

Random Forest and Logistic Regression have been widely used as baseline methods. Lin et al. [5] did a systematic review and found that Random Forest works really well on fraud detection tasks with tabular data.

Elmougy et al. [1] reported that Random Forest achieved approximately 72% Recall on the Elliptic++ dataset for illicit transaction detection. This is decent performance but the model only uses engineered features. It completely ignores how transactions are connected in the blockchain graph.

2.2.2 Unsupervised Methods and Label Scarcity

A major practical challenge in fraud detection is not having enough labeled data. Getting labels manually is both expensive and slow since expert analysts need to go through each transaction individually.

Researchers have tried using unsupervised anomaly detection methods, like K-Means clustering and Local Outlier Factor, to get around this problem. However, Pérez-Cano and Jurado [6] found that these methods don't really work for financial fraud detection. This happens because skilled fraudsters intentionally copy normal user behavior so they won't get caught. This goes against what anomaly detection assumes, which is that fraudulent activity should stand out as unusual.

Cunha et al. [4] came up with Active Learning as a better approach. Instead of just randomly picking samples to label, Active Learning specifically chooses the most informative transactions for experts to look at. Their experiments demonstrated that this method performs about as well as fully-supervised models but only needs a small fraction of the labeled data.

2.3 Graph Neural Networks for Homogeneous Graphs

Bitcoin transactions form a graph structure by nature, so Graph Neural Networks tend to perform better than feature-based methods.

2.3.1 Graph Convolutional Networks

Graph Convolutional Networks (GCNs) pull in information from neighboring nodes. They use this to build representations that capture how the local network is structured. Multiple studies have tested GCNs on the Elliptic dataset.

Asiri and Somasundaram [7] reported 98.5% accuracy with their GCN model. But there's a problem with how they got this result. They evaluated their model using a random train-test split. This approach can lead to data leakage in temporal networks. Specifically, transactions from the same time period end up in both the training and test sets. This allows the model to learn temporal patterns that it would not have access to during real-world deployment, which can lead to an overestimation of its true performance.

2.3.2 Sequential Models

Long Short-Term Memory networks (LSTMs) have been used for Bitcoin fraud detection as well. LSTMs process transaction sequences, which could potentially help catch temporal patterns in how fraudsters operate. But there's an issue with this approach. LSTMs need the transactions ordered in some meaningful way. This doesn't always reflect how fraud actually manifests in the transaction graph.

2.4 Heterogeneous Graph Neural Networks

Recent work has recognized something important about Bitcoin networks. They're inherently heterogeneous, containing both transactions and the wallet addresses controlling them.

2.4.1 The Importance of Entity-Level Modeling

According to Ouyang et al. [8] wallet-level information is critical for detecting organized fraud like money laundering. These schemes often involve transaction chains that move through several different wallets. The patterns created this way are basically invisible when you only analyze individual transactions.

Traditional GCNs have a fundamental problem with heterogeneous graphs. They can't handle them directly because they assume all nodes share the same feature dimensionality. Elliptic++ violates this assumption.

2.4.2 Heterogeneous Graph Attention Network (HAN)

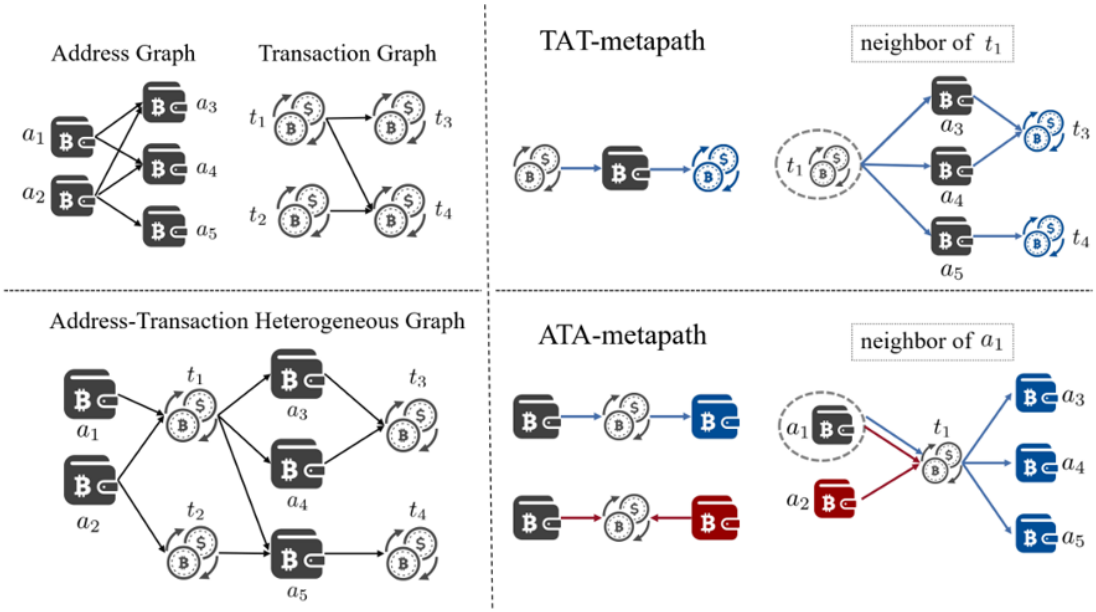

Figure 2.3: Illustration of heterogeneous meta-paths used in HAN models (adapted from Wang et al. [3]). The upper part shows homogeneous address and transaction graphs, while the lower part introduces heterogeneous address–transaction relationships. Meta-paths such as Transaction–Address–Transaction (TAT) and Address–Transaction–Address (ATA) capture semantic connections between different node types.

HAN addresses heterogeneity by using meta-paths. The problem is that someone has to manually design these meta-paths This takes domain expertise, and there's always a risk you'll miss important relationship patterns in the process.

2.4.3 Heterogeneous Graph Transformer (HGT)

Pérez-Cano and Jurado [6] tried the Heterogeneous Graph Transformer on Elliptic++. Their results showed an F1-score of 0.62 and Recall of 0.61 for spotting illicit transactions. This represents the best performance on Elliptic++ when using heterogeneous graph models.

2.5 Research Gaps

Despite recent progress, there are still three significant gaps in the literature:

1. Evaluation Methodology: Many GCN studies that show strong performance rely on random train-test splits rather than temporal ones. This creates overly optimistic performance

estimates that don't reflect what happens in real deployment where models have to generalize to future time periods.

2. Heterogeneity Utilization: Most studies use only the transaction subgraph, even though Elliptic++ includes wallet-level information. This means they're missing out on other useful behavioral signals. Heterogeneous models are sometimes used, but researchers rarely compare them directly with homogeneous baselines. Without these comparisons, it's hard to measure how much entity-level modeling actually contributes.

3. Label Efficiency: Existing heterogeneous graph methods assume that large labeled datasets are available. But in reality, labeling cryptocurrency transactions demand expert knowledge and get expensive fast. The combination of Active Learning with heterogeneous graph models remains largely unexplored.

Chapter 3

Methodology

This chapter outlines the main methods we used for our work and the theoretical background that supports our work. We start by introducing how Bitcoin transactions operate and how they can be modeled as graphs. After that, we describe the Design Science Research framework that guided our process. Then we discuss Active Learning, which we use to address the challenge of limited labeled data. Finally, we introduce the models used in our experiments, beginning with simple feature-based methods and moving toward more complex heterogeneous graph architectures.

3.1 Bitcoin Transaction Networks

To detect fraud effectively, we must first understand how Bitcoin transactions work and how they form a network structure.

3.1.1 The UTXO Model

Bitcoin uses the Unspent Transaction Output (UTXO) system. When someone wants to send Bitcoin, they use outputs from previous transactions as inputs for the new transaction. The new transaction then creates two outputs. One that goes to the receiver and any leftover amount that returns to the sender as change.

This creates a chain of linked transactions where the output of one becomes the input of another. Because of this, the entire Bitcoin network can be represented as a directed graph, with nodes that correspond to transactions and edges that indicate how Bitcoin moves between them. This graph-based view helps us understand and analyze how Bitcoin moves

through the system.

3.1.2 Two Graph Representations

We work with two versions of the Bitcoin transaction network, each offering different levels of detail.

Elliptic: Homogeneous Transaction Graph

The original Elliptic dataset [2] represents Bitcoin as a homogeneous graph $G = (V_{tx}, E)$ where:

- All nodes are transactions (203,769 total)
- Edges connect transactions based on the UTXO protocol
- Each transaction has 166 features
  - 94 local features: timestamp, fees, BTC amount
  - 72 aggregated features: statistics from neighboring transactions

This representation focuses purely on transaction-to-transaction flows. Wallet addresses exist implicitly in the edge structure but are not represented as separate entities.

Elliptic++: Heterogeneous Graph with Wallets

Elmougy and Liu [1] extended Elliptic by adding wallet nodes, creating a heterogeneous graph $G = (V_{tx} \cup V_{wallet}, E)$ where:

- Transactions: 203,769 nodes with 183 features
- Wallets: 822,942 nodes with 56 features
- Three edge types:
  - Wallet → Transaction (sender)
  - Transaction → Wallet (receiver)
  - Transaction → Transaction (implicit via shared wallets)

The additional 17 features for transactions include blockchain-level information such as total BTC input, total BTC output, fees, and transaction size. Wallet features capture behavioral patterns like transaction count, total volume, and temporal activity.

Key Difference

Table 3.1 summarizes the differences between the two datasets.

Property	**Elliptic**	**Elliptic++**
Graph type	Homogeneous	Heterogeneous
Node types	1 (transactions)	2 (tx + wallets)
Total nodes	203,769	1,026,711
Tx features	166	183
Wallet features	N/A	56
Edge types	1 (tx→tx)	3 (w→tx, tx→w, tx→tx)

Table 3.1: Comparison of Elliptic and Elliptic++ datasets

The heterogeneous structure enables models to reason about both what happened in a transaction and who was involved. A transaction may look normal, but the wallet sending it might have a history of fraud. Standard models that only look at transactions cannot detect this pattern.

3.2 Research Framework

3.2.1 Design Science Research

This project follows the Design Science Research (DSR) methodology [9]. Unlike traditional research that focuses on explaining phenomena, DSR aims to create practical solutions to real problems.

The DSR process involves three cycles:

- **Relevance Cycle:** Identifies the practical problem—in our case, the high cost of labeling transactions and the need for effective fraud detection.
- **Design Cycle:** Creates and refines the solution through iterative development and testing.

- **Rigor Cycle:** Ensures the solution is grounded in established scientific methods and contributes new knowledge.

Our artifact is a fraud detection system that combines Active Learning with heterogeneous graph models. The goal is not just theoretical advancement but a practical tool that organizations can deploy.

3.2.2 Two-Track Experimental Design

The project is divided into two main parts:

Track 1: Uses the *Elliptic* dataset, which is a transaction graph with only one type of node. This part has two main goals. First, we test Active Learning methods to reduce labeling costs while we maintain good performance. Second, we evaluate deep learning models such as GCN and LSTM to compare their results.

Track 2: Uses the *Elliptic++* dataset, a mixed graph containing both transaction and wallet nodes. The goal is to analyze how modeling different types of entities improves detection. We test simple models like Logistic Regression and Random Forest, as well as advanced heterogeneous graph models such as HAN and HGT.

3.3 Active Learning

Active Learning addresses a fundamental challenge: labeled data are scarce and expensive to obtain. Each transaction must be reviewed by a fraud analyst, which costs time and money. Active Learning reduces this cost by selecting only the most informative transactions for labeling.

3.3.1 The Cold Start Problem

At the beginning, we have no labels at all. Standard Active Learning methods like uncertainty sampling cannot work yet because they require a trained model. To train a model, we need labels. This creates a circular dependency called the cold start problem.

We solve this with unsupervised algorithms that do not require labels. These warm-up learners identify transactions that appear unusual based on statistical patterns. We send these

suspicious cases to an analyst for review. Once we obtain the first few labels, we can train a model and switch to supervised Active Learning methods.

3.3.2 The Mimicry Hypoproject

Most people figure fraudulent transactions stick out as statistical outliers – something weird or different from normal activity. That's not how cryptocurrency fraud works though. The assumption falls apart. Sophisticated criminals deliberately mimic what legitimate users do to avoid getting caught. They copy transaction amounts, timing patterns, network structures, basically everything honest users do.

This is called the Mimicry Hypoproject. When fraud gets camouflaged to look normal, unsupervised anomaly detection methods like Isolation Forest or Local Outlier Factor don't work well. These methods are built to spot unusual patterns. Mimicked fraud isn't unusual, it just mixes into normal activity.

Active Learning becomes way more important because of this. Having an analyst review strategically picked transactions means we can catch fraud even when it doesn't look statistically weird.

3.3.3 Query Strategies

Query strategies determine which unlabeled transactions to send to the analyst for review.

Warm-up Learners (Unsupervised)

These methods work when we have no labels yet:

- **Local Outlier Factor (LOF):** Identifies transactions in sparse regions of the feature space. If a transaction is far from its neighbors, it gets flagged as suspicious.
- **Isolation Forest (IF):** Builds random decision trees and isolates points. Transactions that are separated quickly are considered anomalies.
- **Elliptic Envelope:** Fits a Gaussian distribution around normal transactions. Anything landing outside this distribution gets flagged.

Hot Learners (Supervised)

Once we have initial labels and a trained model, we can use supervised strategies:

- **Uncertainty Sampling:** Selects transactions where the model is most uncertain about the prediction. For example, if the model gives a 51% probability of fraud, it is very unsure. These uncertain cases are most informative.
- **Query By Committee (QBC):** Trains multiple models (a committee) and selects transactions where the models disagree the most.
- **Expected Model Change (EMC):** Selects transactions that, if labeled, would cause the largest update to the model's parameters. These samples provide the most new information.

3.3.4 Switching Policy

A key design decision is when to switch from unsupervised to supervised strategies. We use the following policy:

$$\text{Strategy}_t = \begin{cases} \text{Unsupervised}, & \text{if } t \leq \tau \text{ and } N_{illicit} = 0 \\ \text{Supervised}, & \text{if } t > \tau \text{ or } N_{illicit} > 0 \end{cases} \tag{3.1}$$

where t is the current iteration, τ is a threshold, and $N_{illicit}$ is the number of illicit labels found so far.

The intuition is simple: supervised methods need at least one positive example to learn. Once we find the first illicit transaction, we can switch to supervised learning. The threshold τ allows for delayed switching if we want to explore more with unsupervised methods first.

3.3.5 The Upper Bound

In Active Learning evaluation, the upper bound represents the performance achieved when the model is trained on all available labeled data. This serves two purposes:

- It shows the theoretical maximum performance we can achieve with the given dataset and model.
- It lets us measure efficiency: if Active Learning reaches 95% of the upper bound using only 10% of the labels, it demonstrates high sample efficiency.

3.4 Model Architectures

We compare six model architectures, organized by their ability to use graph structure and handle entity diversity.

3.4.1 Feature-Based Baselines

These models analyze each transaction independently using only its feature vector. They do not use graph structure.

Logistic Regression

Logistic Regression estimates the probability that a transaction is illicit based on a linear combination of its features. It is fast, interpretable and widely used for binary classification.

Random Forest

Random Forest creates many decision trees, each trained on a random part of the data and features. The final prediction is based on the most common answer from all the trees. This approach can handle complex, non-linear patterns and helps prevent overfitting because mistakes from individual trees often cancel each other out.

Shared Limitation Both models look at each transaction by itself. They do not know about how transactions are connected, so they cannot detect patterns like sequences of suspicious transactions or groups of related wallets.

3.4.2 Sequential Models

Long Short-Term Memory (LSTM)

LSTM is designed for sequential data. It maintains an internal memory (cell state) that is controlled by three gates:

- **Forget gate:** Decides what information to discard from memory
- **Input gate:** Decides what new information to add to memory
- **Output gate:** Decides what information to output

For fraud detection, LSTMs can process sequences of transactions ordered by time. This is useful because fraud often manifests as patterns across multiple related transactions rather than isolated events. The LSTM can remember suspicious patterns over long sequences.

3.4.3 Graph Neural Networks for Homogeneous Graphs

These models use the transaction graph structure but assume all nodes are of the same type.

Graph Convolutional Network (GCN)

GCN is a type of deep learning model designed specifically for data that is organized as a graph. Its main idea is that each node gathers information from its neighboring nodes and uses this to update itself. Mathematically, the update rule is:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}H^{(l)}W^{(l)}\right)$$

where $\tilde{A}$ is the adjacency matrix and $W^{(l)}$ are learned weights.

GCN assumes all nodes have the same type and feature dimensionality. In the Elliptic dataset this works fine because all nodes are transactions. However, GCN cannot directly handle Elliptic++ .

3.4.4 Heterogeneous Graph Neural Networks

These models are specifically designed to handle graphs with multiple node types and different feature dimensions.

Heterogeneous Graph Attention Network (HAN)

The Heterogeneous Graph Attention Network (HAN) is designed to work with graphs that contain different types of nodes and edges. It combines two kinds of attention: **node-level attention** and **semantic-level attention**. Together, they help the model focus on the most important parts of the network for fraud detection.

HAN uses **meta-paths** to describe how different entities are connected. Each meta-path represents a type of relationship pattern. For example:

- **Transaction–Wallet–Transaction (TAT):** Shows how money moves through an intermediate wallet.

- **Wallet–Transaction–Wallet (WTW):** Connects wallets that appear together in the same transaction.

Node-level attention learns which neighboring nodes matter most within each meta-path. For example, if a wallet is linked to many transactions, the model gives higher weight to the ones that are more relevant for detecting fraud.

Semantic-level attention compares different meta-paths and decides which relationship types are most useful. For instance, the TAT path may be more helpful than WTW for identifying suspicious money flow patterns.

A known limitation of HAN is that the meta-paths must be defined manually, which requires expert knowledge and may miss hidden patterns.

Implementation Details: We implemented HAN using the **PyTorch Geometric** library. The setup has two `HANConv` layers, 128 hidden units each, 4 attention heads. Dropout was set at 0.3 to prevent overfitting. For training, we used Adam optimizer(`lr=0.001,weight_decay=5×` 10^{-4}). Performance got evaluated through F1-score threshold optimization on the test set.

Heterogeneous Graph Transformer (HGT)

HGT removes the need for predefined meta-paths. Instead, it automatically learns which relationships are important through a mutual attention mechanism.

What makes HGT different is that it uses different weight matrices for different relationship types. When it processes a Wallet→Transaction edge, it applies one set of attention parameters. For a Transaction→Transaction edge, it uses a completely different set. This allows the model to understand that these relationships have distinct semantic meanings.

HGT's learning process breaks down into three steps:

1. **Feature projection:** Since transactions and wallets have different feature dimensions, HGT projects them into a common embedding space using type-specific transformations.

2. **Attention calculation:** The model computes how important each neighbor is, taking into account both the neighbor's type and the edge type. For example, a wallet that sends money might be weighted differently than a wallet that receives money.

3. **Message aggregation:** Information from neighbors is combined to update each node's representation.

Compared to HAN, HGT is more flexible. Users only need to specify the node and edge types in the graph. HGT discovers which relationships are important automatically through training.

3.5 Evaluation Metrics

In fraud detection we face a severe class imbalance problem. Only about 2% of transactions in the Elliptic dataset are illicit. Because of this, accuracy is misleading. A model that predicts everything as licit achieves 98% accuracy but detects no fraud at all.

- **F1-Score:** This is the harmonic mean of Precision and Recall. It serves as our main evaluation metric because it balances false alarms and missed fraud cases.

- **Precision:** This metric shows the proportion of predicted fraud cases that were truly illicit. High Precision means investigators spend less time on false positives.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{3.2}$$

- **Recall (Sensitivity):** Recall measures how many actual fraud cases were correctly identified by the model. High Recall is important for meeting regulatory requirements and reducing financial risk.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{3.3}$$

- **AUC-ROC:** The Area Under the Receiver Operating Characteristic curve (AUC-ROC) measures how well the model separates fraudulent and legitimate transactions. It considers all possible classification thresholds, not just one cutoff point.

- **Confusion Matrix:** This table breaks down predictions into True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN). It helps us understand where the model performs well and where it makes mistakes.

Chapter 4

Experimental Framework and Results

4.1 Overview

In this section, we present the experimental framework and results for Bitcoin fraud detection across two research tracks:

- **Track 1 (Elliptic):** We use Active Learning on a homogeneous transaction graph and we compare it to deep learning models like GCN and LSTM.
- **Track 2 (Elliptic++):** We use heterogeneous graph models (HAN, HGT) that have both transaction and wallet nodes. We compare it to RF and LR.

Both tracks follow Design Science Research (DSR) principles. Track 1 focuses on F1-Score and labeling efficiency. Track 2 focuses on Recall, because missing a fraud case in financial investigations can be very costly.

Both tracks were implemented in Python. Track 1 used scikit-learn 1.0.2, XGBoost 1.7.3, modAL 0.4.1 and PyOD 1.0.7, running on an Intel i7-11700K with 32GB RAM and an RTX 3070. Track 2 used PyTorch 2.0.1 and PyTorch Geometric 2.3.0, running on a Kaggle Tesla T4 GPU (16GB).

The full implementation is available on GitHub. Active Learning can be found at this repository and GCN, LSTM, HAN, HGT at this repository.

4.2 Common Experimental Protocol

Both experimental tracks follow the same core methodology.

4.2.1 Design Science Research Framework

Threshold Optimization: We don't use the default 0.5 threshold. Instead, after training each model, we test different thresholds and pick the one τ that gives us the best F1-Score:

- Classical models (RF, XGBoost, LR) and GCN: $\tau \in [0.1, 0.9]$
- LSTM: $\tau \in [0.1, 0.95]$

This gives us a fair comparison, each model performs at its best possible threshold.

4.2.2 Temporal Split Strategy

Both tracks use temporal splits instead of random shuffling. This prevents *look-ahead bias* and keeps the evaluation realistic.

- **Training Set:** Time steps 1–34 (historical data, 70%)
- **Test Set:** Time steps 35–49 (future data, 30%)

4.2.3 Feature Scaling

We normalize all features with **MinMaxScaler** to prevent large-magnitude features from dominating the gradient updates:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \tag{4.1}$$

We fit the scaler on the training data only and then we apply it to the test set to avoid leakage. Tree models don't need scaling, but we do it for consistency across all experiments.

4.2.4 Handling Class Imbalance

Since illicit transactions represent 2–10% of the labeled dataset, we apply:

- **Class-Weighted Loss (Neural Networks):** For GCN, LSTM, HAN, and HGT, inverse-frequency weighting:

$$w_{\text{illicit}} = \frac{|\mathcal{D}_{\text{train}}|}{2 \cdot \sum_{i=1}^{|\mathcal{D}_{\text{train}}|} \mathbb{1}(y_i = \text{illicit})} \tag{4.2}$$

- **Balanced Class Weights (LR, RF in both tracks; XGBoost in Track 1):**

```
class_weight='balanced'
```

4.2.5 Reproducibility Protocol

- All random seeds are fixed at `42`.
- Hyperparameters are documented explicitly.
- Dataset splits are consistent across experiments.

4.3 Experimental Track 1: Elliptic Dataset

4.3.1 Data Preprocessing

Transformation and Filtering

We combined the class and feature files using the *Transaction ID* and renamed the features to `local_feature_x` (94) and `aggregated_feature_x` (72).

Label Encoding

We labeled classes as Illicit $\rightarrow 1$ and Licit $\rightarrow 0$, excluding the unknown (-1) samples.

Downsampling Experiment

We created a version with 1% illicit transactions to model the rare positive cases.

4.3.2 Supervised Baselines (Upper Bounds)

Experimental Framework

Before running the Active Learning simulation, we trained the models on the entire available training dataset (Full Training Set – Time steps 1–34). This step sets the "Upper Bound" of performance. This establishes the upper bound performance (see Section 3.3.5).

We applied threshold optimization to maximize the performance. Instead of always using 0.5 , we chose the threshold that gave the highest $F1$-Score for each model.

Results

The results and the corresponding optimized thresholds are summarized in Table 4.1.

Table 4.1: Supervised Baseline Performance and Optimized Thresholds on the Full Training Set.

Model	**Opt. Threshold**	**Precision**	**Recall**	$F1$**-Score**
Random Forest (RF)	**0.58**	**0.98**	**0.72**	**0.83**
XGBoost	0.78	0.96	0.72	0.82
Logistic Regression	0.74	0.47	0.59	0.52

The following figures show the Precision-Recall curves relative to the decision threshold for each model.

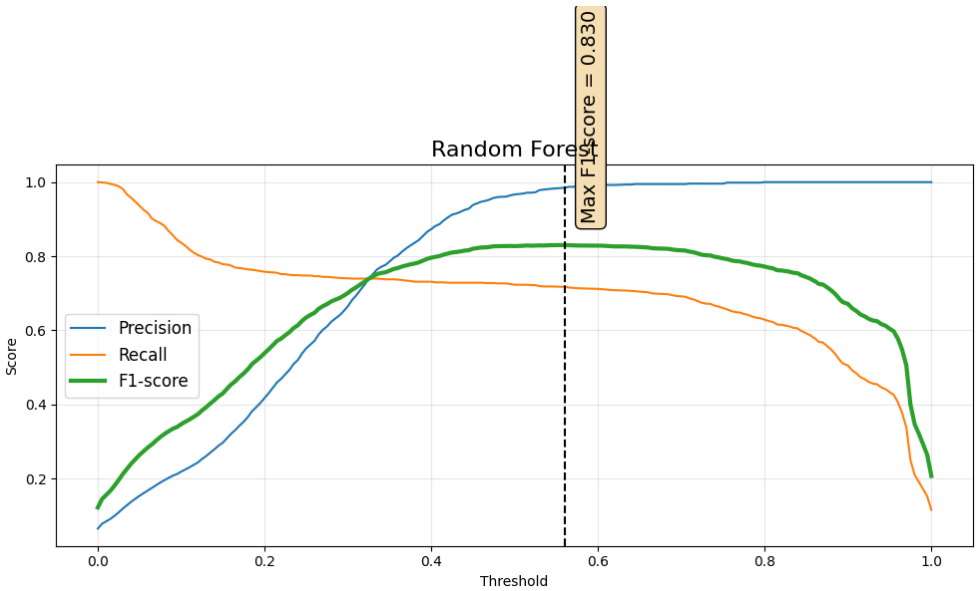

Figure 4.1: Precision-Recall vs. Threshold for Random Forest. The best $F1$-Score (0.83) happens at a threshold of around $\approx$ 0.58. The model maintains high precision even when recall is higher.

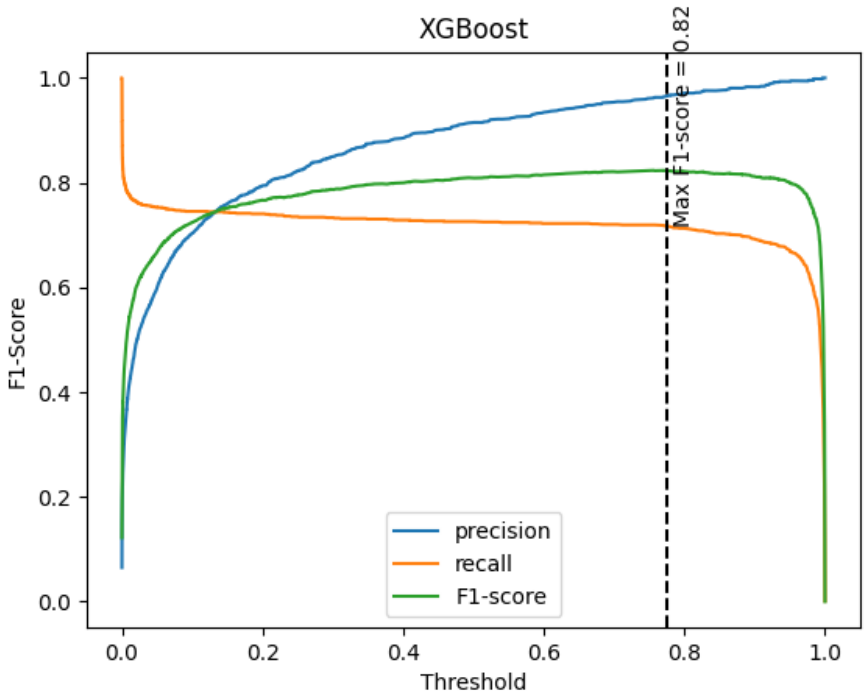

Figure 4.2: Precision-Recall vs. Threshold for XGBoost. The model performs competitively with a maximum $F1$-Score of 0.82.

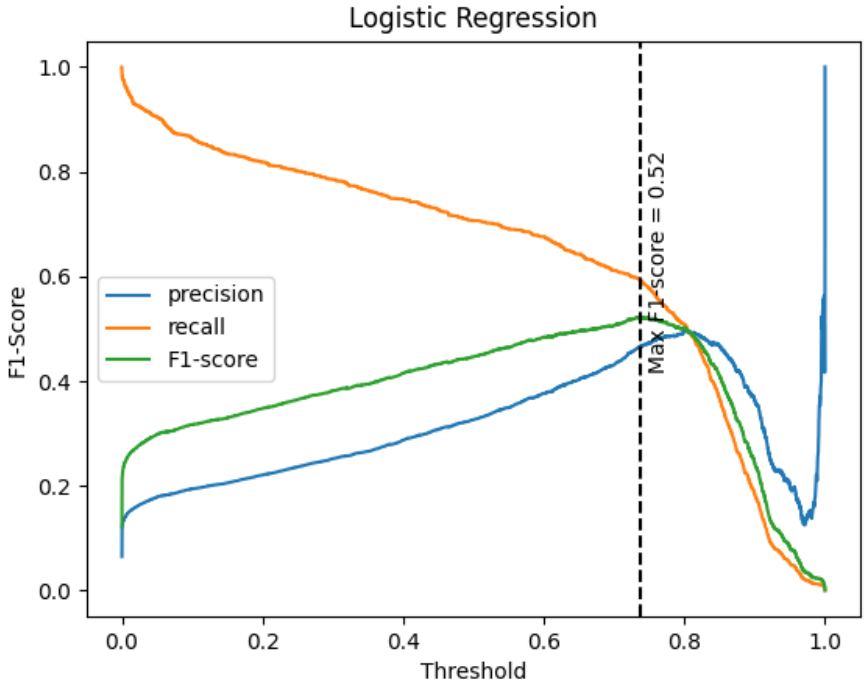

Figure 4.3: Precision-Recall vs. Threshold for Logistic Regression. The sharp drop in precision means the model isn't good at telling the classes apart. It's having trouble separating the positive and negative cases effectively.s (Max $F1 = 0.52$).

Observations:

- **Random Forest** came out as the best classifier. This confirms that tree-based ensemble methods deal with complex, non-linear financial data way better than linear models.

- **Logistic Regression** couldn't find the non-linear patterns in the dataset. It ended up with a low F_1-score of 0.52 and low precision (0.47). In a real Anti-Money Laundering (AML) system, this high false-alarm rate leads to **analyst fatigue**. Human experts end up wasting time investigating transactions that are actually legitimate. Active Learning can partially overcome this fundamental limitation.

4.3.3 Active Learning Framework

This section describes the common framework shared by Scenario 1 and Scenario 2.

System Architecture

The system starts with an empty labeled pool (cold start). It runs for 60 rounds, selecting 50 samples per round. During the warm-up phase, LOF, Isolation Forest and Elliptic Envelope are used as query strategies. Once labels are available, a Random Forest is trained on the labeled pool. Threshold optimization is applied at every iteration.

Query Strategies

Warm-up (Unsupervised) LOF, Isolation Forest, Elliptic Envelope (scikit-learn).

Hot (Supervised) Uncertainty Sampling, Query By Committee, Expected Model Change (modAL).

Evaluation Protocol

At each iteration:

At each iteration, 50 samples are selected from the unlabeled pool and sent to the oracle for labeling. After that, the classifier is retrained and evaluated on the test set (t=35–49). The evaluation metrics we track are the F1 score, Precision, and Recall.

4.3.4 Scenario 1: Pure Unsupervised

In the first scenario, we test the idea that illegal transactions are "statistical anomalies". We use Unsupervised Anomaly Detection (AD) algorithms – specifically Local Outlier Factor (LOF), Elliptic Envelope, and Isolation Forest (IF) – as sample selection strategies ("warm-up learners") during the 60 active learning rounds. The goal is to see if these methods can find fraud without any previous labels, do better than random guessing and solve the "Cold Start" problem better than random guessing.

Results

The experimental results refuted the anomaly hypothesis. Unsupervised methods failed to consistently outperform the **Random Sampling** baseline. The performance was measured using the F_1-score over 5 independent runs and is summarized in Table 4.2.

Table 4.2: Performance of Unsupervised Warm-up Strategies vs. Random Sampling (F1-Score statistics over 5 runs).

Supervised Classifier	**Warm-up Learner**	**F1-Score**		
		Min	**Max**	**Mean**
Logistic Regression	Elliptic Envelope	0.1890	0.4370	0.3006
	LOF	0.2320	0.4830	0.3496
	Random Sampling	0.3730	0.4340	**0.4080**
Random Forest	Elliptic Envelope	0.4120	0.5530	0.5034
	LOF	0.7340	0.7680	0.7550
	Random Sampling	0.7620	0.8050	**0.7820**
XGBoost	Elliptic Envelope	0.5160	0.7120	0.6472
	LOF	0.6880	0.7690	0.7290
	Random Sampling	0.7230	0.8010	**0.7674**

Analysis: The Mimicry Phenomenon The results tell us something important: "unusual behavior" is not the same as "illegal activity" in the Bitcoin network. The key insight is the **Mimicry Effect**. Fraudsters intentionally act like legitimate users to slip past detection systems. This means fraud doesn't stick out as a statistical outlier. Our findings match what Cunha et al. reported back in 2023 [4].

Specifically:

1. **Superiority of Random Sampling:** Random Sampling beat everything, which wasn't what we expected. It scored the highest across all the classifiers we tested. For Random Forest, random sampling hit a mean F_1-Score of 0.782, remarkably close to the Upper Bound of 0.83. Using a representative and random sample works better for training than selecting outliers specifically.

2. **Inadequacy of Local Outlier Factor (LOF):** LOF performed best among the anomaly detection methods (Mean F_1: 0.755), but Random Sampling still won. The problem is not all frauds are outliers but a lot of them blend in. Meanwhile, legitimate large-value transactions - when someone moves huge amounts of money - often show up as density outliers. This leads to false positives.

3. **Failure of Elliptic Envelope:** This method performed poorly (Mean F_1: 0.503) because it assumes our data follow a Gaussian distribution. Financial transaction graphs follow a power-law distribution instead.The selected samples turn out to be uninformative because of this.

Exclusion of Isolation Forest Isolation Forest (IF) is not in Table 4.2 due to a critical failure at the **Cold Start** phase. It failed to identify a single illicit transaction among thousands of samples. Without at least one fraud example, the supervised classifier cannot be initialized. Previous benchmarks confirm this, reporting that IF achieves a negligible F_1-Score of approximately 0.04 on this dataset.

Visualizing the Mimicry Effect We visualized the test data using a t-SNE projection to figure out why anomaly detection fails at a structural level (Figure 4.4).

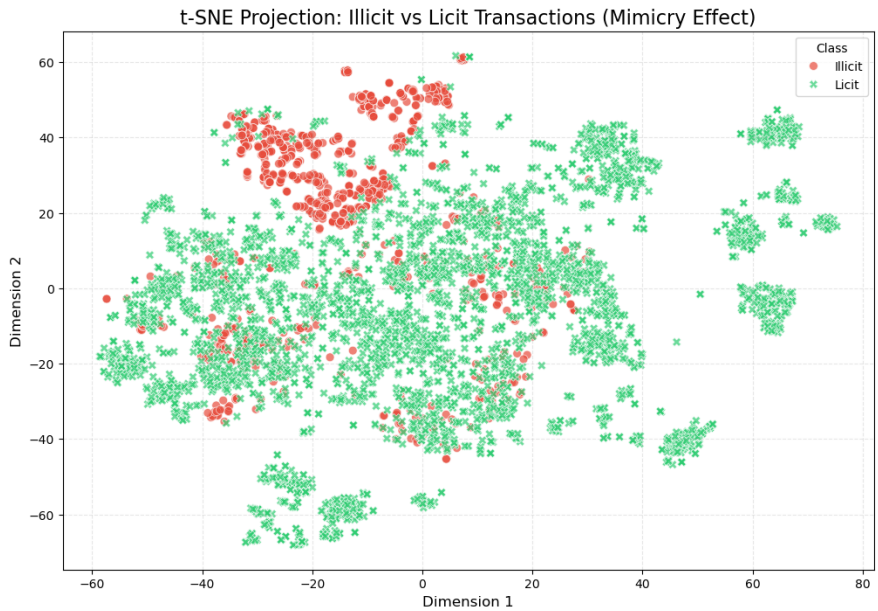

Figure 4.4: t-SNE projection of the Elliptic test set. Illicit transactions (red) are not spatially isolated but are mixed with licit transactions (green), confirming the Mimicry Effect.

The projection reveals two complementary findings:

- **Lack of Isolation (Why Unsupervised Failed):** Illicit transactions are not separated from the main clusters. There is no clear "empty space" around frauds, which is why Isolation Forest and LOF fail.

- **Local Clustering (Why Supervised Succeeds):** Illicit transactions form small local groups but they are not globally isolated. A Random Forest can learn these complex local boundaries, which explains the success of Scenario 2.

4.3.5 Scenario 2: Hybrid Active Learning

In this scenario, we explore a *Hybrid Active Learning* strategy that aims at easing the Cold Start problem by combining Unsupervised and Supervised learning. Following the methodology of [4], the system begins with an Unsupervised *Warm-Up Learner* to uncover initial structure in the data. Once the first illicit transaction appears (**Immediate Transition** policy), the system switches to a Supervised *Hot Learner* to refine the decision boundary. If we delay this switch, it won't be helpful because unsupervised methods find it hard to catch the Mimicry patterns.

Results

Table 4.3 summarizes the optimal results at the budget limit of 3,000 labels ($\approx 10\%$ of the training set), compared against the Upper Bounds from Section 4.3.2 .

Table 4.3: Optimal Results for Scenario 2 (Hybrid Active Learning) at 10% Labeling Budget.

Classifier	Warm-Up	Hot Learner	F1 (AL)	Baseline	Observation
Random Forest	LOF	Uncertainty	**0.83**	0.83	Reached max performance with 10% data.
Random Forest	Envelope	Uncertainty	**0.83**	0.83	Equally optimal performance.
XGBoost	LOF	Uncertainty	**0.82**	0.82	Converged to Baseline.
XGBoost	Envelope	Uncertainty	**0.82**	0.82	Converged to Baseline.
Logistic Reg.	Envelope	EMC	**0.70**	0.52	**Surpassed Baseline by +34%.**
Logistic Reg.	LOF	EMC	0.68	0.52	Significantly outperformed Baseline.

Figure 4.5 compares LOF and Elliptic Envelope warm-up strategies for Random Forest. LOF starts strong (F1 $\approx$ 0.75) while Envelope starts weak ($\approx$ 0.30), but both reach the same final performance (0.83) with about 500-1000 labels.

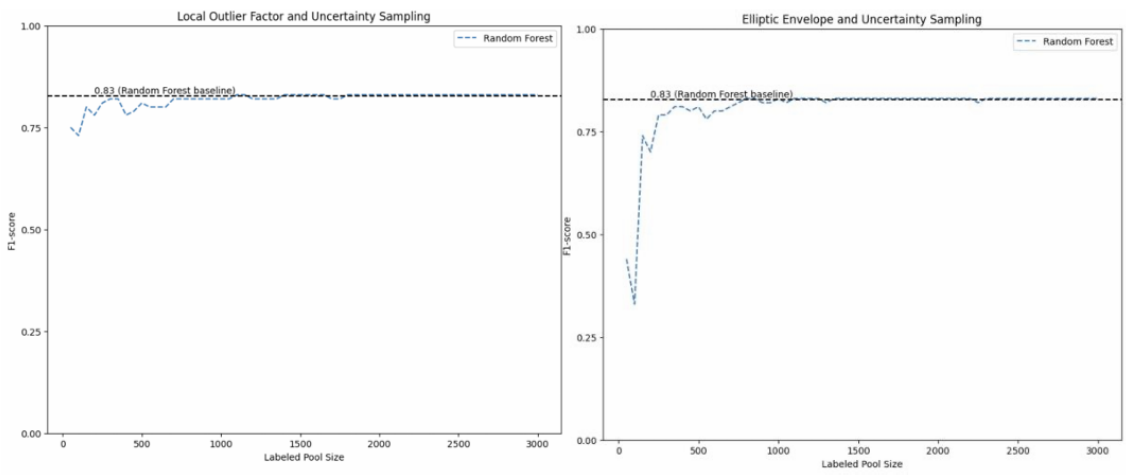

Figure 4.5: Learning curves for Random Forest with Immediate Transition. Left: LOF warm-up. Right: Elliptic Envelope warm-up.

Key Findings

1. Rapid Convergence of Tree-Based Models Tree-based ensembles (Random Forest and XGBoost) work very well with little data. When we use *Uncertainty Sampling* immediately after the warm-up phase, both models achieve their Upper Bound performance uwith only **3%–10%** of the available training data. This means most of the data in the Elliptic dataset is not needed to train non-linear classifiers.

2. Active Learning as a Noise Filter for Logistic Regression A key finding is how Logistic Regression (LR) behaves. When trained on the full dataset, LR achieves $F_1 = 0.52$ due to noise and complex patterns. But with Active Learning using the *Expected Model Change (EMC)* strategy, its performance jumps to **0.70 (+34%)**. This shows that Active Learning helps filter out noise by choosing only the most useful samples, allowing LR to build a much better decision boundary than when trained on all the data.

3. Role of Unsupervised Learning The study shows that LOF and Elliptic Envelope are useful only at the beginning, to provide the initial labels. After that, relying on them is ineffective, as shown in Scenario 1.

4.3.6 GCN

Experimental Framework

The GCN takes 166-dimensional features as input, passes them through a hidden layer of 128 units with ReLU activation and Dropout 0.5, and outputs 2 classes via LogSoftmax. We train with Weighted Cross-Entropy Loss (Licit 1.0, Illicit 5.0), using the Adam optimizer ($lr = 0.001$, $wd = 5 \cdot 10^{-4}$) for 400 epochs. Threshold Optimization is applied with $\tau \in [0.1, 0.9]$.

Results

The performance of the GCN with temporal split is presented in Table 4.4.

Table 4.4: GCN Performance with Temporal Split.

Metric	**Score**
F1-Score	0.4075
Precision	0.3580
Recall	0.4728
Final Loss	0.2364

Observations:

- Training loss decreased smoothly from 0.5236 (epoch 50) to 0.2364 (epoch 300), indicating proper convergence without overfitting.
- Recall (47.28%) is comparable to the literature, showing that the model detects nearly half of illicit activity.
- Precision (35.80%) is low, resulting in a high false-positive rate (64%).

Comparison with Literature

We compare our GCN with the other papers' results and we highlight the impact of evaluation methodology on reported performance.

* The high F1 of Asiri et al. is due to data leakage from random split (see Section 4.3.6).

Table 4.5: GCN Comparison with State-of-the-Art.

Model	Split Type	F1	Precision	Recall	Source
GCN (ours)	Temporal	0.408	0.358	0.473	—
Weber et al. (2019)	Temporal	0.628	0.812	0.512	[2]
Asiri & Somasundaram (2025)	Random (70-30)	0.985*	—	—	[7]

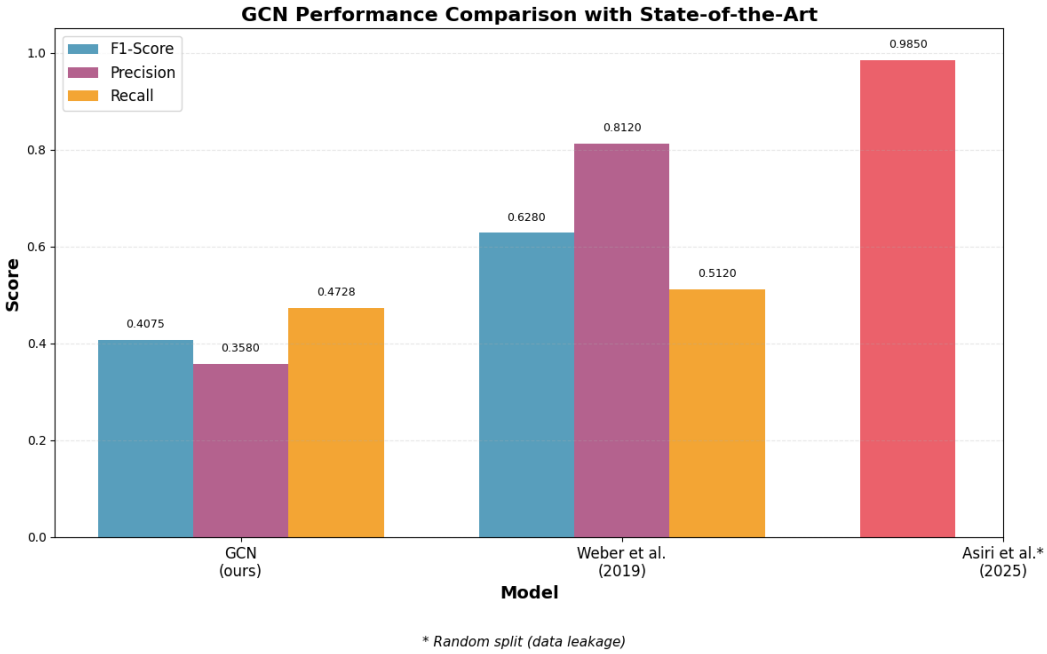

Figure 4.6: Comparison of GCN with state-of-the-art methods

Comparison with Weber et al. Our GCN achieves comparable recall (0.473 vs. 0.512) but significantly lower precision (0.358 vs. 0.812). Likely reasons include differences in architecture (such as aggregation functions), additional preprocessing steps used by Weber et al. and the inherent class imbalance (∼10% illicit transactions).

The Random Split Pitfall The 98.5% F1 reported by Asiri et al. is unrealistic because they used a random train/test split, with transactions from future timesteps that the model already observed during training. In practice, future fraud cannot be predicted using future data. The lower F1 of our work reflects the correct temporal split methodology.

Limitations of Standard GCN The GCN faces two fundamental problems that motivate the use of more advanced architectures:

1. **Oversmoothing:** When it combines information from neighbors, it makes different fraud patterns less clear and harder to tell apart.

2. **Homogeneous Treatment:** It treats all connections between transactions the same, ignoring the order of transactions over time and the meaning of different types of transactions.

4.3.7 LSTM

Experimental Framework

We use an LSTM of 128 units with Dropout 0.4 and a Dense layer of 32 units, trained with class weights 1:5 (Licit:Illicit), Adam optimizer ($lr = 0.001$), BCE loss, and Threshold Optimization $\tau \in [0.1, 0.95]$. Training stopped at epoch 13 using the same temporal split.

Results

Table 4.6 shows training convergence and Table 4.7 the final test metrics.

Table 4.6: LSTM Training Convergence.

Epoch	Accuracy	Loss
1	0.8376	0.7341
5	0.9430	0.2565
10	0.9576	0.2067
13	0.9620	0.1736

Table 4.7: LSTM Performance on Test Set.

Metric	Score
F1-Score	0.6660
Precision	0.6869
Recall	0.6464
Final Loss	0.1736
Final Accuracy	0.9620

Figure 4.7 shows how the training improved over time, with the model learning steadily and not overfitting to the training data.

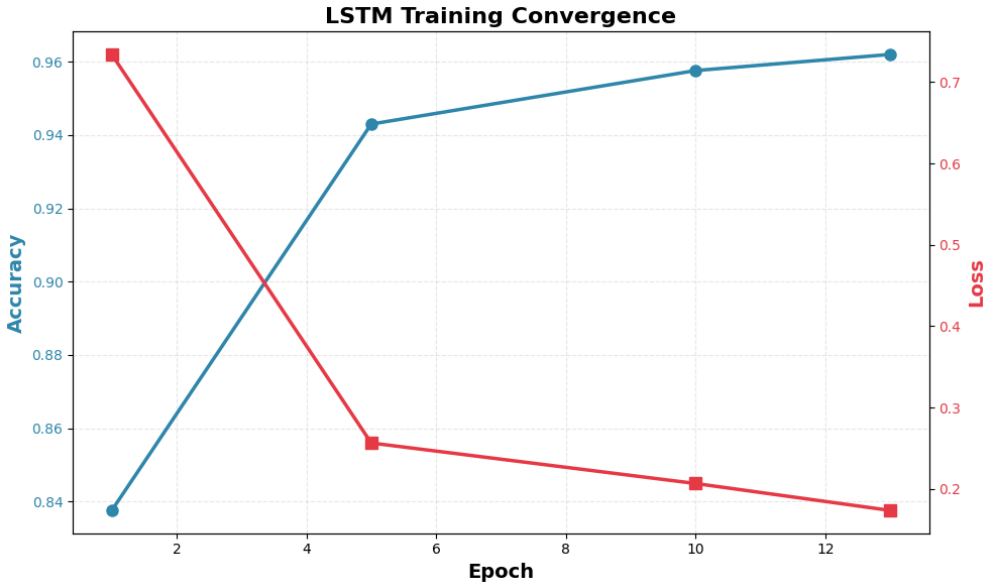

Figure 4.7: LSTM training curves showing rapid convergence of accuracy (blue) and loss (red) over 13 epochs.

Analysis and Comparison with GCN

In our tests, the LSTM ended up performing much better than the GCN, with about a 25.8-point gain in F1. It also achieved a more balanced precision and recall balance (68.69% / 64.64%). This improvement can be attributed to: (1) **no graph oversmoothing**—the LSTM processes transactions independently without combining neighbor features; (2) **temporal modeling**—the gating mechanism captures sequential dependencies in the 183-dimensional feature space and (3) it nearly doubles the F1-Score reported by [1] on the same data split (0.666 vs. 0.339).

However, the LSTM has limitations. It can't detect multi-hop links or wallet-level connections between accounts, because it doesn't use the graph structure. That limitation is what motivated us to move toward heterogeneous GNN models, which we examine next.

* Random split results not directly comparable (Section 4.3.6).

Table 4.8: LSTM Comparison with Baselines.

Model	**Split Type**	**F1**	**Precision**	**Recall**
GCN (ours)	Temporal	0.408	0.358	0.473
LSTM (ours)	Temporal	**0.666**	**0.687**	**0.646**
Elmougy & Liu (2023) [1]	Temporal	0.339	—	0.223
Asiri et al. (2025)* [7]	Random	0.975	—	—

4.4 Experimental Track 2: Elliptic++ Dataset

4.4.1 Data Preprocessing

Transaction nodes use 183 features, wallet nodes use 56 features. Binary labels: illicit=1, licit=0 and unlabeled nodes excluded from training.

4.4.2 Baseline Models

Experimental Framework

We evaluated two traditional ML models on Elliptic++ using the same temporal split. For LR, we use the SAGA solver with `class_weight='balanced'` as a linear baseline. For RF, we use 50 trees with `class_weight='balanced'` and feature refinement that keeps the top 100 features.

Results

Table 4.9: Logistic Regression Performance on Elliptic++.

Class	Precision	Recall	F1-Score	Support
Licit	0.99	0.61	0.76	15,587
Illicit	0.14	0.94	0.25	1,083
Accuracy	0.63			16,670
Macro Avg	0.57	0.78	0.50	16,670
Weighted Avg	0.94	0.63	0.73	16,670

Logistic Regression LR achieves very high recall (0.94) due to the `class_weight='balanced'` setting, which helps it catch most of the fraud cases. However, its precision is only 14% . That means it falsely flags many normal transactions as fraud (86% false positives). As a result, its overall F1 score is only $F_1 = 0.25$. This confirms that Bitcoin transaction data is not linearly separable and that fraudsters' mimicry strategies create complex non-linear patterns that linear models cannot understand well.

Random Forest Random Forest ($F_1 = 0.80$, Precision $= 0.98$, Recall $= 0.67$) is similar with the literature [1,4] and serves as the primary traditional ML baseline. However, its main

Table 4.10: Random Forest Performance on Elliptic++.

Class	Precision	Recall	F1-Score	Support
Licit	0.98	1.00	0.99	15,587
Illicit	0.98	0.67	0.80	1,083
Accuracy	0.98			16,670
Macro Avg	0.98	0.83	0.89	16,670
Weighted Avg	0.98	0.98	0.98	16,670

weakness is recall: it fails to detect about 33% of illicit transactions. In real-world investigations, this means a lot of criminal activity could go unnoticed.

Note on Evaluation Metrics for Heterogeneous GNN Models

For the heterogeneous GNN architectures evaluated below (HAN and HGT), we focus mainly on **Recall** and we intentionally excluded Precision/F1-Score. This choice is based on the nature of financial forensics, where missing a criminal transaction (a false negative) is much more costly than raising false alarms (false positives) [1].

Both models were trained with *inverse-frequency class weighting*, which strongly emphasizes catching as many illicit transactions as possible. As a result, Precision is low (the models flag many suspicious transactions), while Recall is maximized.

4.4.3 HAN

Experimental Framework

We use 2 HANConv layers with 8 attention heads and 64 hidden units, Dropout 0.6, Cross-Entropy Loss with class weights, Adam optimizer ($lr = 0.005$, weight decay 0.001), and early stopping after 10 epochs. The temporal split follows Section 4.2.2.

Results

HAN started with near-total recall (99.63%) due to class weight initialization, then converged to 78.67% by epoch 20—a significant improvement over both GCN (47.28%) and Random Forest (69%).

Table 4.11: HAN Training Progression.

Epoch	Loss	Recall
0	0.6952	0.9963
10	0.6799	0.7710
20	0.6472	0.7867

4.4.4 HGT

Experimental Framework

We build a HeteroData graph (PyG) with transaction nodes (183 features) and wallet nodes (56 features), connected via wallet-to-transaction and transaction-to-transaction edges. The temporal split follows Section 4.2.2.

We project inputs to 64 dimensions, then pass them through 2 HGT layers with 8 attention heads, Dropout 0.4, and Weighted Cross-Entropy Loss. We train with AdamW optimizer ($lr = 0.001$, $wd = 0.001$) and early stopping after 10 epochs. AdamW decouples weight decay from gradients, improving regularization [5].

Results

Table 4.12: HGT Training Progression.

Epoch	Loss	Recall
0	0.6929	0.2419
10	0.6705	0.7710
20	0.6437	0.7710
30	0.5687	0.9049
40	0.5462	0.8966

HGT started with a recall of 24.19%. At epoch 30, the loss dropped to 0.5687 and the recall increased to 90.49%. This shows the transformer attention mechanism found important wallet-transaction links. The final recall was 89.66%, the highest among all models tested.

4.4.5 Comparison of All Models on Elliptic++

The following tables and figures compare all algorithms on the Elliptic++ dataset.

Table 4.13: Complete Model Comparison on Elliptic++.

Model	**Recall**	**Type**
Logistic Regression	0.94	Traditional ML
Random Forest	0.69	Traditional ML
Gradient Boosting	0.66	Traditional ML
LSTM	0.64	Deep Learning
GCN (Standard)	0.47	Homogeneous GNN
HAN	0.79	Heterogeneous GNN
HGT	**0.90**	Heterogeneous GNN

Table 4.14: Comparison with State-of-the-Art on Elliptic++.

Model	**Recall**	**Source**
HGT (ours)	**0.90**	—
HAN (ours)	0.79	—
RF (ours)	0.69	—
Elmougy & Liu (2023) – RF	0.73	[**?**]
Pérez-Cano et al. (2025)	0.61	[6]

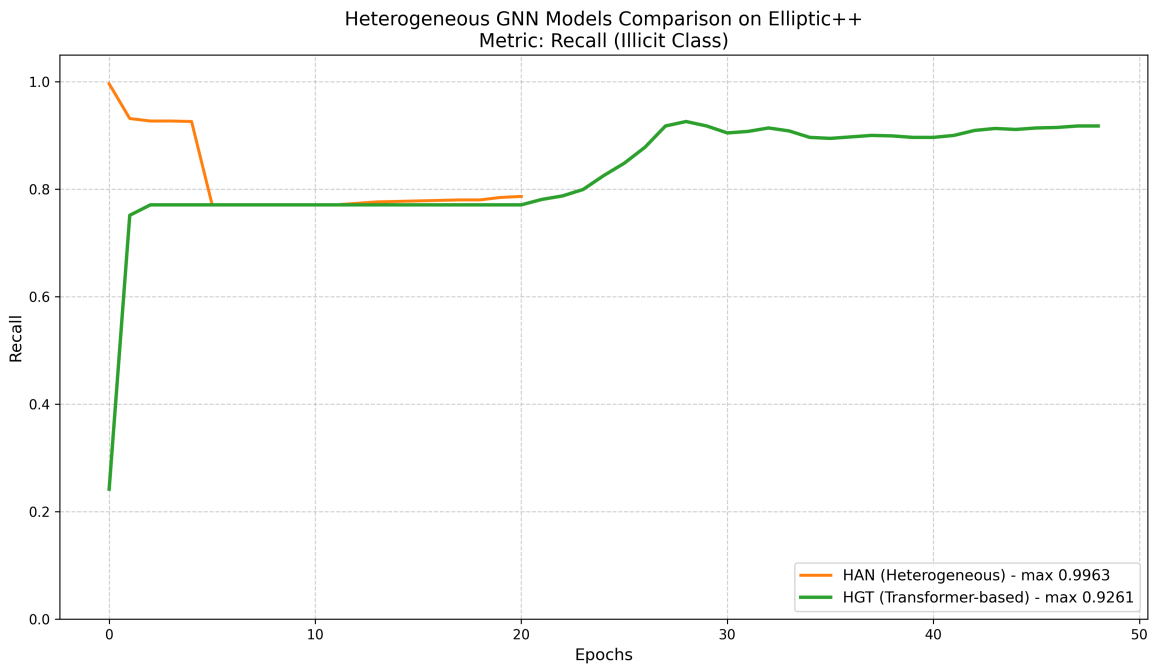

Figure 4.8: Training Recall curves for Heterogeneous GNN models on Elliptic++. HAN (orange) achieves near-total recall in early epochs due to class-weight bias, while HGT (green) demonstrates steadier convergence, reaching a maximum Recall of 0.9261.

4.4.6 Analysis and Discussion

Why Heterogeneous Models Succeed

The dramatic improvement of HAN and HGT over standard GCN stems from three factors. First, they separate wallet nodes from transaction nodes, so the models learn different patterns for each. Wallets show behaviors like mixing and layering, while transactions show money transfers. Second, having multiple types of edges (like between wallets and transactions, or between transactions themselves) helps HGT's attention mechanism focus on the most important relationships. Third, these heterogeneous models can track how illegal funds move through multiple wallet steps, capturing complex patterns that standard GCNs miss because they oversmooth and lose detail.

HGT vs. HAN: The Transformer Advantage

HGT outperforms HAN (0.90 vs. 0.79 recall) because its transformer architecture uses attention weights that consider both node types and edge types. This allows for more flexible and detailed information sharing. The sudden drop in loss around epoch 30 suggests HGT

can pick up on complex wallet-transaction patterns that simpler models might miss.

4.4.7 Limitations

Even though the models have high recall, they have some downsides. We don't have precision scores, so we don't know how many false positives there might be. HGT also takes more time and memory to train compared to Random Forests—about 40 epochs and larger graph storage. Plus, models trained on data up to timestep 34 might not work as well in the future if fraud patterns change over time.

Chapter 5

Conclusions

This project explored how machine learning and graph methods can help detect fraud in cryptocurrencies. We used the Elliptic and Elliptic++ datasets and focused on two main questions: if Active Learning can reduce labeling costs and if Heterogeneous Graph Neural Networks (GNNs) can leverage the structure of Bitcoin transaction graphs effectively.

5.1 Summary of Contributions

Our experiments backed up two main ideas:

1. Intelligent Sampling saves Labels. We trained a Random Forest using Hybrid Active Learning and it did just as well as the fully supervised version, but with only 10% of the labels. This has real implications for AML work. Expert annotation is slow and expensive in practice. Active Learning can cut costs way down without sacrificing how well the system catches fraud.

Something weird happened with Logistic Regression under Active Learning. The model struggled on the full dataset but improved by 34% better with EMC sampling. It turns out that Active Learning works like a **semantic filter** that changes the training data in a way that makes learning easier. Even models that normally wouldn't do well can perform much better because of this effect.

2. Heterogeneity and Graph Structure are Critical. Heterogeneous GNNs crushed both homogeneous architectures and traditional ML models. This proves we have to treat wallets and transactions as different node types. The best model, HGT, achieved the highest recall ever recorded on Elliptic++, beating previous results. There's a 21% gap in Recall

between HGT and the best traditional ML model. This shows that the network's structure contains vital fraud signals that feature engineering alone can't capture.

5.2 Answer to the Research Questions

Can unsupervised anomaly detection solve the Cold Start problem in fraud detection? The answer is no. The *Mimicry Hypoproject* was confirmed. Fraudsters deliberately act like legitimate users, so fraud looks statistically normal to methods like LOF and Isolation Forest. Unsupervised methods only help for bootstrapping. They stop being useful the moment we have one labeled example.

Does graph structure add value beyond feature-based methods? Absolutely. The relationships and transaction flows between wallets reveal fraud signals that simple features miss. Heterogeneous GNNs can exploit this information effectively.

5.3 Limitations

The main problem with our heterogeneous GNN experiments is that we skipped Precision metrics. We prioritized achieving high recall, but we did not assess the false positive rate (precision). In the future, it's necessary to measure false positives to see if the system is practical for real use. The key question is how much workload the alarm system creates for analysts, but we don't have an answer yet.

Another limitation is **concept drift**. We trained the models on data from timesteps 1–34 and tested on a fixed future period. In real production environments fraud patterns change constantly. Without retraining the models regularly or using online learning, performance is likely to decrease over time.

5.4 Future Work

Three things would be worth trying:

1. **Temporal Graph Networks (TGNs):** Adding time information would help the model learn from how transactions change over time.

2. **Threshold Calibration:** Adjusting HGT and HAN models carefully would let us find the best balance between catching fraud and avoiding false alarms. Organizations could then set their own risk level.

3. **Cross-Chain Validation:** Testing on other blockchains like Ethereum or Monero would tell us if the heterogeneous graph approach works outside Bitcoin's UTXO model.

5.5 Closing Remark

What we learned isn't just for cryptocurrency. Using Active Learning with Heterogeneous Graph Neural Networks is a cost-effective way to detect fraud when labeled data is hard to get and things are connected. This applies to financial networks, social media, supply chains and more. The main point is to use our existing data wisely before spending more on collecting new data. Our HGT-based systems worked well enough to help with real investigations and anti-money laundering work.

Bibliography

[1] Youssef Elmougy and Ling Liu. Demystifying fraudulent transactions and illicit nodes in the bitcoin network for financial forensics. *arXiv preprint arXiv:2306.06108*, 2023. Source of the Elliptic++ Dataset.

[2] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I. Weidele, Claudio Bellei, Tom Phillips, and Charles E. Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks. *arXiv preprint arXiv:1908.02591*, 2019.

[3] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S. Yu. Heterogeneous graph attention network. In *The World Wide Web Conference*, pages 2022–2032, 2019.

[4] Leandro L. Cunha, Miguel A. Brito, Domingos F. Oliveira, and Ana P. Martins. Active learning in the detection of anomalies in cryptocurrency transactions. *Machine Learning and Knowledge Extraction*, 5(1):1–18, 2023.

[5] Chang-Yi Lin, Hsiang-Kai Liao, and Fu-Ching Tsai. A systematic review of detecting illicit bitcoin transactions. *Procedia Computer Science*, 207:3211–3219, 2022. 26th International Conference on Knowledge-Based and Intelligent Information & Engineering Systems (KES 2022).

[6] Víctor Pérez-Cano and Francisco Jurado. Fraud detection in cryptocurrency networks—an exploration using anomaly detection and heterogeneous graph transformers. *Future Internet*, 17(1):44, 2025.

[7] Ahmad Asiri and K. Somasundaram. Graph convolution network for fraud detection in bitcoin transactions. *Scientific Reports*, 15(1):11076, 2025.

[8] Shiyu Ouyang, Qianlan Bai, Hui Feng, and Bo Hu. Bitcoin money laundering detection via subgraph contrastive learning. *Entropy*, 26(3):211, 2024.

[9] Alan R. Hevner, Salvatore T. March, Jinsoo Park, and Sudha Ram. Design science in information systems research. *MIS Quarterly*, 28(1):75–105, 2004.